SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT133

Timed Online Assignment - July Semester 2022

Structured Programming

Thursday, 17 November 2022 04:00 pm - 06:00 pm

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR** (**4**) questions and comprises **SIX** (**6**) pages (including cover page).

2. You must answer **FOUR (4)** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. **Images of handwritten answers will not be marked**.

6. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

7. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

ICT133 Copyright © 2022 Singapore University of Social Sciences (SUSS)

Answer all questions. (Total 100 marks)

Question 1

Express a sequence of statements based on computational logic.

Question 1a

Write Python statements for each of the following tasks. Do not use collection structures (str, list, dict, etc) and loop structures.

(i) A coin is fair if the probability of getting head is between 0.45 and 0.55, inclusive of both numbers at both ends. A coin is tossed multiple times, resulting in `n1` heads and `n2` tails. Write one or more statements to evaluate whether the coin is fair based on `n1` heads and `n2` tails, and set the result of the evaluation to a bool variable, `fair`.

(ii) Given a sting `s`, print `SAME` if the letter `a` appears the same number of times in the both halves of `s`, print `MORE` if it appears more times in the first half of `s`, print `FEWER` if it appears fewer times in the first half of `s`. If the length of `s` is odd, do not include the middle letter in the two halves, so that the lengths of both halves are exactly the same.

For example, the string `s` is `bananas`, and its length is odd. So, the middle character <u>a</u> in `ban`<u>`a`</u>`nas` is not included. `SAME` is printed as the letter `a` appears the same number of times in the first half `ban` and the second half `nas`.

(10 marks)

Question 1b

Write Python statements for the following tasks. You may use collection structures restricted to str, list and dict. You may also use loop structures.

Question 1b(i)

Write a code segment to process a string `s1` to produce another string `s2` without repeating immediate neighbouring characters. If `s1` is empty, `s2` is set to empty also. For example, when processing a string `mississippi`, the code segment produces the string is `misisipi`.

(5 marks)

Question 1b(ii)

Write a code segment to determine whether a word `w2`, e.g., `love` can be made from another word `w1`, e.g., `solve`. Print `Possible` if it is possible and `Impossible` if it is not possible. For example, it is not possible to make the word `live` from `solve` but it is possible to make the word `love` from `solve`.

(6 marks)

Question 1c

Use an appropriate segment of code written in Q1(a) and Q1(b) to explain why an expression is a type of building block in program.

(4 marks)

ICT133 Copyright © 2022 Singapore University of Social Sciences (SUSS)

Question 2

Apply data structures to store and process information, and write Python code for each task.

Question 2a

`staffSalaryList` is a list of elements where each element is a list of two elements – staff number and salary for a staff. Staff numbers are unique in `staffSalaryList` but salaries are not unique.

(i) Update the salary for the first element in `staffSalaryList` to 2500.

(ii) Locate the salary of staff with staff number `p123`. If the staff can be located, update salary to 3600 and print `Updated!` If staff cannot be located, print `Unable to locate`.

(iii) Compute the average salary of staff. Print the staff number whose salary is above the average salary.

(iv) Print the staff number in descending order of salary and in ascending order of staff number, that is, print the staff number of the staff with the highest salary first, and if the salaries tie, print the smaller staff number first. Do not print the salary.

(10 marks)

Question 2b

Process the same data in `staffSalaryList` in Q2(a) now recorded in a dictionary `staffSalaryDict` with staff number as key and salary as value.

(i) Initialise `staffSalaryDict` using the data in `staffSalaryList`.

(ii) Determine and print the number of staff number and salary pairs there are in `staffSalaryDict`.

(iii) Locate the salary of staff with staff number `p123`. If the staff can be located, update salary to 3600 and print `Updated!` If staff cannot be located, print `Unable to locate`.

(iv) Print the staff number and salary of staff in ascending order of salary and in ascending order of staff number if the salaries tie.

(11 marks)

Question 2c

Based on the operations in Q2(a) and Q2(b), explain which operations are best suited for list and which operations are best suited for dictionary. Use the operations in Q2(a) and Q2(b) in your explanation.

(4 marks)

ICT133 Copyright © 2022 Singapore University of Social Sciences (SUSS)

Question 3

Apply data structures to store and process information, and solve computational problems using structured programming.

The rates (in cents) for sewing alphabets onto clothing are shown in Figure Q3(a). For example, the cost of sewing an uppercase `H` is 45 cents and a lowercase `a` is 63 cents.

A 44	B 79	C 60	D 56	E 46	F 50	G 65	H 45	I 38
J 49	K 51	L 36	M 48	N 55	O 56	P 53	Q 41	R 58
S 65	T 36	U 23	V 49	W 46	X 48	Y 54	Z 52	
a 63	b 55	c 37	d 55	e 63	f 58	g 40	h 31	i 35
j 50	k 51	l 36	m 32	n 58	o 50	p 55	q 55	r 53
s 61	t 57	u 37	v 33	w 59	x 53	y 54	z 39	

Figure Q3(a)

Question 3a

Describe how you will store the data shown in Figure Q3(a). Explain why you have chosen to store the data in that way.

(4 marks)

Question 3b

Write a function `computeCost` that has a string parameter `lines` and one or more parameters for the data in Figure Q3(a), depending on your answer to Q3(a). The function returns the cost (in dollars) of printing only alphabets appearing in `lines`. Cents not making up to 5 cents should be removed.

For example, the cost of sewing `'Ha\nHa\nHa'` is (45+63)x3 = 324 cents or $3.24. The function removes 4 cents and returns 3.2. If the cost is 3.27, the function returns 3.25 after removing 2 cents.

(11 marks)

Question 3c

Write a function `justifyLines` that has one string parameter `lines`. The function returns a formatted string with the lines in `lines` center-justified to the longest line in `lines`. If there are uneven numbers of spaces to pad a line, put the extra space in front of the line. For example, `'Ms\nLovely\Heart'` should be aligned as `'  Ms  \nLovely\n Heart'`.

(10 marks)

Question 4

Employ structured programming principles to develop a program.

Question 4a

Implement a function, **`checkSequence`**. The function has one string parameter containing either integers and one or more dashes, separated by commas (,). The string starts with an integer, not necessarily 1.

The function returns `True` if a sequence of consecutive integers can be obtained from the string when the dashes are replaced by integers. The function returns `False` otherwise. For example, suppose the string value is:

- `'2,-,-,5,6,-,-'`

The function returns True because when the dashes are replaced by 3, 4, 7 and 8, a sequence of consecutive integers `'2,3,4,5,6,7,8'` is formed.

- `'1,2,-,-,6,-,-,-'`

The function returns `False` because when the first two dashes are replaced by 3 and 4 and the last three dashes are replaced by 7, 8 and 9, the string `'1,2,3,4,6,7,8,9'` does not form a sequence of consecutive integers,

(6 marks)

Question 4b

Read a file, data.txt which contains one or more lines, each line containing either integers and one or more dashes, separated by commas (,). An example content is as follows:

```
2,-,-,5,6,-,-
1,2,-,-,6,-,-,-
```

Process each line in the file by calling the function **`checkSequence`** defined in Q4(a), and appending the result of each call on separate lines in a file, output.txt.

(4 marks)

Question 4c

Implement a function, **`generateSequence`** which has an integer parameter `length`. The function generates and returns a string consisting of `length` unique integers and dashes, separated by commas (,). The string starts with an integer, not necessarily 1. If `length` is 0 or negative, the function returns an empty string. If length is 1, the function returns a string of one integer.

An example string is `'2,3,-,-,7,8,9,-,11'` when `length` is 9. Note that the string also satisfies these conditions:

- Integers are ordered in increasing order and neighbouring numbers are consecutive, e.g., 2 and 3 are consecutive and 7, 8 and 9 are consecutive.
- Integers immediately after a dash are at least one but not two more than the sum of the last integer before the dash and the number of dashes, e.g., the number

after the first 2 dashes can be 6 or 7, and 7 is generated and included, instead of 6. Also, the number after the last dash can be 11 or 12, and 11 is generated and included, instead of 12.

(10 marks)

Question 4d

Write a function, **`main`** which repeatedly

- prompts for the length of sequence to generate,
- calls the functions **`generateSequence`** and **`checkSequence`**, and
- prints the output of both function calls

until the user enters **`0`** when prompted for the length. If the user enters negative numbers, display an error message and re-prompt the user.

<u>Sample run:</u>

```
Enter length of sequence to generate and 0 to end: -2
Length of sequence must be a positive number
Enter length of sequence to generate and 0 to end: 10
89,90,91,92,93,-,96,97,-,- : False
Enter length of sequence to generate and 0 to end: 10
83,84,85,-,87,-,89,90,-,- : True
Enter length of sequence to generate and 0 to end: 9
5,6,7,8,9,-,11,12,- : True
Enter length of sequence to generate and 0 to end: 0
```

(5 marks)

----- END OF PAPER -----